

Narrative Template Design

What it is

This concept lives in one file: `bilingual_etl/transform/narrative_builder.py`. Its job is to take a company's (or category's) structured data — fields like `year_founded: 1996`, `nr_employees: 13`, `legal_form: "Korlátolt felelősségű társaság"` — and turn them into a paragraph of ordinary, readable prose, in either Hungarian or English, depending on which language row is being built.

This is **R6**: the requirement that the system generate a natural-language narrative per language, explicitly *not* a keyword dump or a JSON blob. Two structural rules come with that requirement:

1. Category and activity lists must appear near the **end** of the narrative, not dumped at the top.
2. Corporate facts (legal form, employee count, founding year, and so on) must be rendered as full sentences — never as `key: value` pairs like `year_founded: 1996`.

The module itself makes **zero LLM calls**. It is a pure, deterministic templating function: same input fields in, same sentence out, every time, no network calls, no randomness, and it runs in microseconds rather than the seconds an LLM call would cost. Everything it needs — including which language a field's free text is already written in — has been decided by an earlier stage of the pipeline before this module ever runs.

Real-life analogy

Picture two different ways a real-estate agent could hand you information about a house for sale.

Version one is a spreadsheet row: `year_built: 1998`, `sqft: 1400`, `bedrooms: 3`, `garage: yes`. Technically it's all the facts you asked for. But nobody reads a listing like that and feels anything — it doesn't tell a story, and if you tried to search a huge database of these rows using natural language ("a cozy older three-bedroom with a garage"), a spreadsheet full of `key: value` pairs is a bad match for that kind of question. The words "cozy," "older," and "garage" barely overlap with `sqft: 1400`.

Version two is what a good listing description actually reads like: "Built in 1998, this inviting three-bedroom home offers 1,400 square feet of living space and a private garage." Same underlying facts, but now they're woven into sentences a human — or, as it turns out, a machine that's been trained on human language — can actually relate to.

`narrative_builder.py` is the machine that turns version one into version two, automatically, for every company in the dataset, in two languages, without ever needing a human real-estate agent (or an LLM) to sit down and write each one by hand.

Why we used it here

The reason prose matters isn't just aesthetics — it's what happens *after* this narrative leaves this module. The narrative text becomes the exact string that gets sent to an embedding model (see

docs/learning/06-embeddings-and-provider-pattern.md), which converts it into a vector for semantic search.

Embedding models are trained on enormous amounts of natural human writing — books, articles, web pages, conversation. They are very good at understanding what "a company founded in 1996 with 13 employees" *means*, semantically, because that's the kind of sentence they've seen billions of times during training. They are comparatively bad at understanding what `year_founded: 1996, nr_employees: 13` means, because raw field-dump text like that barely resembles anything in their training data. Feed an embedding model a JSON blob or a field dump, and the resulting vector tends to be a blurry, low-information point that doesn't sit anywhere near where a human's *semantic* question about the company would land. Feed it a fluent sentence, and the vector lands somewhere much more meaningful — which is exactly what makes semantic search actually work later, downstream, in the Search API.

So this module exists as the bridge between "structured facts we have" and "natural language an embedding model can actually understand" — and it has to do that for every company, twice (once per language), without an LLM call per record, because that would be enormously slow and expensive at scale (471 companies × 2 languages, and growing).

Code walkthrough

The three-tier field-handling strategy

The module docstring lays out the core design decision explicitly — not every field can be "translated" the same way, so the code splits fields into three tiers based on how open-ended their vocabulary is.

Tier 1 — free/open-vocabulary fields. `description` and `activities` (and `category_names`, passed in separately since it comes from a join against the categories dataset). These fields can contain literally any sentence a human wrote — there's no fixed list of possible values. The module does **not** translate these itself. It assumes the caller has already translated them into the target language using `bilingual_etl/enrichment/translator.py` (built in an earlier phase) before calling `build_company_narrative()`. This is a deliberate separation of concerns: translation is a whole subsystem with its own caching and LLM-provider logic; this module's only job is templating, not translation.

Tier 2 — small closed-vocabulary fields. `legal_form`, `company_status`, and `services`. Look at the actual lookup tables in the file:

```
LEGAL_FORM_LABELS = {
    "hu": {
        "Korlátolt felelősségű társaság": "korlátolt felelősségű társaság (Kft.)",
        "Betéti társaság": "betéti társaság (Bt.)",
        ...
    },
    "en": {
        "Korlátolt felelősségű társaság": "Limited Liability Company (Kft.)",
        ...
    },
}
```

Across all 471 companies in the real dataset, `legal_form` only ever takes **7 distinct values** — Hungarian legal entity types are a small, fixed, legally-defined set (Kft., Bt., Kkt., Rt., sole proprietor, and two foreign-representation forms). `company_status` has only 2 values in practice ("Működő" / active, "Bírósági eljárás alatt" / under court proceedings), and `services` is drawn from a fixed list of 9 categories (engineering, installation, logistics, manufacturing, rental, retail, service, webshop, wholesale). Calling an LLM to translate a value from a set this small and this fixed would be wasteful — the same handful of inputs would be sent to a paid API over and over, for an answer that never changes. A plain Python dictionary lookup is the "simplest robust solution": instant, free, and — because it's an explicit table someone can read and edit — trivially auditable if a translation ever needs correcting.

The lookup itself goes through one small helper:

```
def _translate_label(value: str, mapping: dict, language: str) -> str:
    labels = mapping.get(language, {})
    if value not in labels:
        logger.warning(f"No {language} label for value {value!r}; using original text.")
        return value
    return labels[value]
```

If a value ever shows up that isn't in the table (a new legal form appears in the source data, say), this doesn't crash the whole ETL run — it logs a `WARNING` and falls back to using the raw, untranslated text. That's a defensible middle ground: better to surface an ugly-but-present value and a log line an engineer can go fix, than to silently drop the field or hard-crash the pipeline over one unmapped string.

Tier 3 — proper-noun-like fields. `brands` and `certificates`. These are included completely unchanged, in either language:

```
brands = company.get("brands") or []
if brands:
    joined = _join_natural(brands, language)
    if language == "hu":
        sentences.append(f"Forgalmazott márkák: {joined}.")
    else:
        sentences.append(f"Brands they distribute: {joined}.")
```

Note that the *surrounding sentence* ("Forgalmazott márkák:" / "Brands they distribute:") changes per language, but the brand names themselves ("Piranha", "ABS", "ISO 9001") never do — a brand name or a certificate code means the same thing in every language, the same way "Coca-Cola" or "ISO 9001" wouldn't be translated in an English sentence either.

The Hungarian definite article: `_hu_article()`

```
HU_VOWELS = set("aáééííoóöüüü")
```

```
def _hu_article(word: str) -> str:
    """'A' before a consonant, 'Az' before a vowel — e.g. 'Az ABS Kft.', 'A Zultzer Kft.'"""
    return "Az" if word and word[0].lower() in HU_VOWELS else "A"
```

Hungarian's definite article works like English "a" vs. "an": "a" before a consonant sound, "an" before a vowel sound. Hungarian does the same thing with "A" vs. "Az" — "A Zultzer Kft." reads naturally, but "A

ABS Kft." sounds as wrong to a Hungarian speaker as "a apple" sounds to an English speaker.

`_hu_article()` checks the first character of the company (or category) name against a set of Hungarian vowels and picks the grammatically correct article.

A real bug this caught during development: an earlier version of this function returned the literal string "(z)" as a suffix hint rather than choosing between "A" and "Az" outright, which produced sentences like "A(z) Vakolható..." — grammatically broken text that no Hungarian speaker would ever write, sitting right at the start of every single narrative. It was caught and fixed by testing against the real dataset rather than a couple of hand-picked toy examples. That distinction matters here specifically because this isn't a rare edge case: **96 of the 471 real companies** in this dataset have names starting with a vowel (e.g. "ABS Kft.", "AEROCIKLON Kft."). That's roughly **1 in 5 companies** — if this function had shipped broken, one out of every five Hungarian narratives in the whole dataset would have opened with a grammar mistake. A simplistic manual check against two or three example names ("Zultzer", "Test Kft.") would never have surfaced this, because none of those examples happen to start with a vowel. Only running the function against the *entire* real company list exposed it.

Natural list-joining: `_join_natural()`

```
def _join_natural(items: list[str], language: str) -> str:
    if not items:
        return ""
    if len(items) == 1:
        return items[0]
    conjunction = "és" if language == "hu" else "and"
    return f"{' '.join(items[:-1])} {conjunction} {items[-1]}"
```

A naive `" ".join(items)` on `["ABS", "Piranha", "HST"]` produces "ABS, Piranha, HST" — grammatically that's a list, not a sentence. `_join_natural()` instead produces "ABS, Piranha and HST" (or "ABS, Piranha és HST" in Hungarian): every item gets a comma except the last one, which gets the language's conjunction word instead. This one small helper is what turns every list-shaped field in the narrative (activities, services, certificates, brands, categories) from a comma-dump into a proper English or Hungarian sentence fragment.

Assembling the full narrative: `build_company_narrative()`

The function builds up a `sentences` list, one entry per fact, and joins them at the end with `" ".join(sentences)`. Walking through the order:

1. **Opening sentence** — legal form, founding year, employee count, all folded into one sentence per language (using `_hu_article()` and `_translate_label()` for the legal form). This is the "full sentence, not key: value" requirement in action.
2. **Status sentence** — company status, translated via the closed-vocabulary lookup.
3. **Description** — the free-text field, inserted as-is (already translated by the caller).
4. **Activities, services, certificates, brands, counties** — each rendered as its own sentence, using `_join_natural()` for the list fields.

5. Category list — deliberately last. The code comment says it outright:

```
# Category list – deliberately placed near the end, per R6.
if category_names:
    joined = _join_natural(category_names, language)
    if language == "hu":
        sentences.append(f"A cég a következ█ kategóriákban aktív: {joined}.")
    else:
        sentences.append(f"The company is active in the following categories: {joined}.")
```

This ordering matters for the same reason a news article puts the headline before the fine print: whatever comes first in a narrative tends to carry the most weight, both for a human skimming it and for how an embedding model represents the text. Corporate identity and description come first because that's what makes a company distinct; the (potentially very long) category list comes last so it doesn't overwhelm everything else in the narrative before it even gets going.

Real output, from the real dataset

Running the actual function against the real record for **"Zultzer Pumpen Kft."** (a 13-employee pump manufacturer founded in 1996, taken from

`data/companies_data_for_vectors_sample_2026_03_11.json`) produces this Hungarian narrative (category list truncated here for readability — the real output continues for 625 category names before the doc's closing sentence, "...Zégergy█ és Zsinórgy█."):

A Zultzer Pumpen Kft. egy korlátolt felel█sség█ társaság (Kft.), amelyet 1996-ban alapítottak, és jelenleg 13 f█t foglalkoztat. A cég jelenleg m█köd█ státuszú. A Sulzer globális szolgáltatásszállító a szennyvízkezelési technológia terén; teljes termékpalettáján ABS gyártmányú szivattyúkat, kever█ket, leveg█ztet█berendezéseket, kompresszorokat, valamint vezérléseket és felügyeleti eszközöket, illetve szervizt kínál. A Sulzer a vízmentesít█ piac számára is biztosít szivattyúkat. F█ tevékenységi területeik: kompresszor, szennyvíztechnika, szivattyú és tömítéstechnika. Kínált szolgáltatások: mérnöki szolgáltatás, bérlet, beszerelés, nagykereskedelem, szerviz és gyártás. Tanúsítványaik: ISO 14001 és ISO 9001. Forgalmazott márkák: ABS, ABS MF, ABS XFP, HST, Piranha, Pumpex, Scaba, Scanpump és Swedmeter. Lefedett megyék száma: 20. A cég a következ█ kategóriákban aktív: Aktív szenes s█rített leveg█ sz█r█, Autó kompresszor, Axiál-kompresszor, ... [619 more category names] ..., Zégergy█ és Zsinórgy█.

Measured, real word count: 1,553 words total — of which everything up through "Lefedett megyék száma: 20." is only 97 words. The remaining ~1,456 words are the single category-list sentence (see `docs/learning/10-text-chunking-strategies.md` for why that matters for chunking).

The equivalent English narrative (built by feeding the same function pre-translated English text, plus a shortened 6-item category sample for readability in this doc — a real English row would carry all 625 translated category names) looks like this:

Zultzer Pumpen Kft. is a Limited Liability Company (Kft.) founded in 1996 with 13 employees. The company currently has active status. Sulzer is a global service provider in wastewater treatment technology, offering a full range of ABS-manufactured pumps, mixers, aeration equipment, compressors, as well as control and monitoring devices, and servicing. Sulzer also supplies pumps for the dewatering market. Their main areas of activity include: compressors, wastewater technology, pumps and sealing technology. Services offered: engineering, rental, installation, wholesale, servicing and manufacturing. They hold the following certifications: ISO 14001 and ISO 9001.

Brands they distribute: ABS, ABS MF, ABS XFP, HST, Piranha, Pumpex, Scaba, Scanpump and Swedmeter. They cover 20 counties. The company is active in the following categories: Active carbon compressed air filter, Car compressor, Axial compressor, Submersible sewage pump, Sewage shaft and Rubber seal ring.

Comparing the two confirms every design point at once: corporate facts read as full sentences in both languages (not "year_founded: 1996"), the category list sits at the very end in both, and the brand names — ABS, Piranha, HST, Pumpex — are byte-for-byte identical in the Hungarian and English versions, exactly as the proper-noun tier intends.

How to explain this in an interview/to a teammate

"We needed to turn structured company records into natural-language text, because that text goes straight into an embedding model for semantic search — and embedding models are trained on human writing, not JSON. A field dump like 'year_founded: 1996' embeds poorly; a sentence like 'founded in 1996' embeds well, because it looks like the kind of text the model actually learned from. So we built a pure templating function — no LLM calls, deterministic, fast — that assembles each company's facts into prose, per language. We split fields into three tiers based on vocabulary size: free-text fields like the description are pre-translated upstream and dropped in as-is; small closed-vocabulary fields like legal form or company status go through a static lookup table, since there are only a handful of possible values and an LLM call would be overkill; and proper-noun fields like brand names are left untouched in every language, because 'Piranha' means 'Piranha' no matter what language the sentence is written in. We also had to handle Hungarian grammar correctly — picking 'A' vs 'Az' depending on whether the next word starts with a vowel — and testing against the full real dataset, not just a couple of hand-picked examples, is what caught a bug where a leftover placeholder was producing 'A(z) Vakolható' instead of correct Hungarian; nearly one in five real company names in our dataset starts with a vowel, so that wasn't a rare edge case at all."